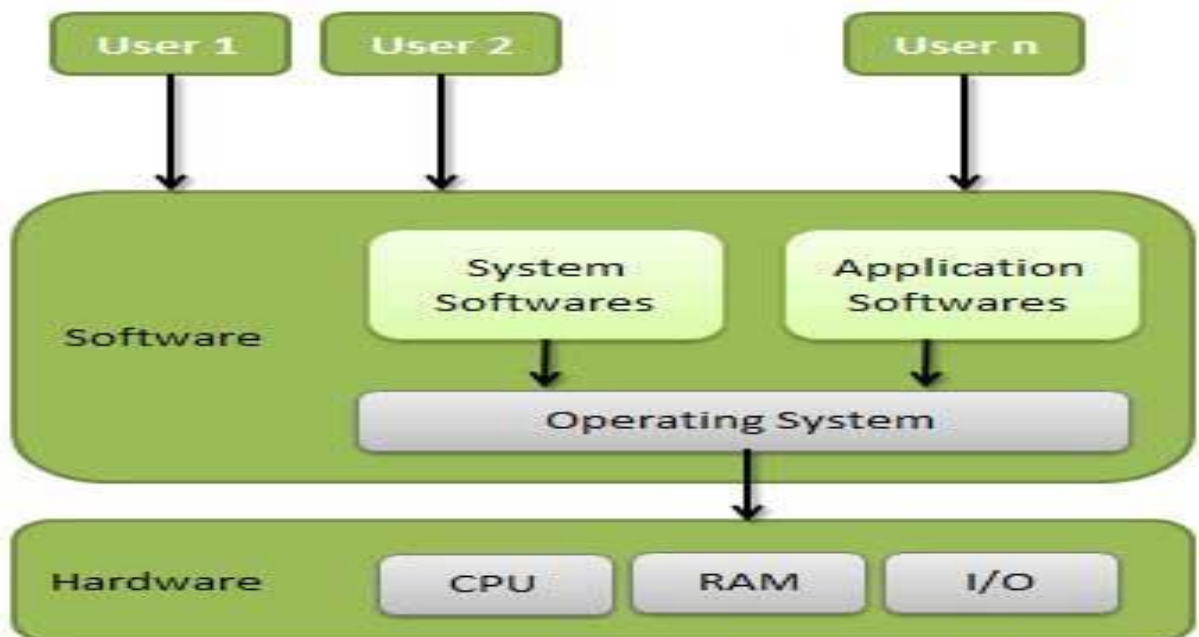


Chapter 1: Introduction to Operating System

1. Definition of Operating System (OS)

- An Operating System (OS) is a system software that acts as an interface between a user and computer hardware.
- It manages computer hardware, provides a user interface, and facilitates execution of programs.
- It handles key resources: CPU, memory, input/output devices, and storage.
- Without an OS, a user cannot communicate with the hardware directly.
- It provides necessary services like process scheduling, file management, and device control.
- It ensures resource sharing and system security.
- Examples: Windows, Linux, macOS, Android.
- It enables multitasking, allowing several applications to run simultaneously.
- Every device (PC, phone, server) has its own OS based on usage needs.
- Thus, OS makes the system functional, secure, and user-friendly.

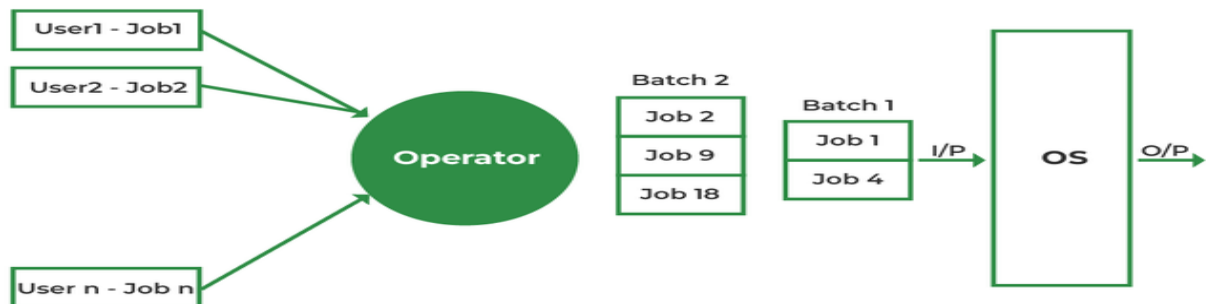


2. Objectives of Operating System

- Convenience: Makes the system easier for the user to use.
- Efficiency: Manages system resources optimally to maximize performance.
- Development: Supports easy testing and addition of new features.
- Resource Utilization: Ensures maximum use of CPU, memory, and I/O.
- Security: Provides mechanisms to protect data and programs.
- User Interaction: Offers a friendly interface via GUI or CLI.
- Program Execution: Handles loading and running of software programs.
- Communication: Manages data exchange between programs or systems.
- Error Handling: Detects and handles system faults.
- File Management: Controls data storage and access.

3. Batch Operating System

The users of batch operating system do not interact with the computer directly. Each user prepares his job on an off-line device like punch cards and submits it to the computer operator. To speed up processing, jobs with similar needs are batched together and run as a group. Thus, the programmers left their programs with the operator. The operator then sorts programs into batches with similar requirements.



Main Components:

Job Queue: Stores jobs waiting to be processed.

Batch Monitor: Special program that manages execution.

Input Device: Typically punch cards or magnetic tape.

Output Device: Line printers, magnetic tape, etc.

Working of Batch OS:

Job Submission:

- User gives the job to the **operator**, not directly to the computer.
- Jobs are usually on **punch cards** or **tape**.

Job Grouping:

- The operator **groups jobs** into a batch based on similarity.

Job Queueing:

- Jobs are loaded into the **job queue** for processing.

Execution Begins:

- The **Batch Monitor** starts executing jobs **one by one**.
- Each job is processed **automatically** from start to end.

Output Stored:

- Output is stored or printed after each job finish. (Line Printer or Tape)

No User Interaction:

- Users **do not interact** with the job while it is running.

Next Job:

- Once one job is done, the next starts immediately.

Advantages:

- Efficient for large, repetitive jobs.
- Reduces idle time of CPU.
- Good for environments with similar, long-running tasks.

Disadvantages:

- No interaction during execution.
- Difficult debugging (errors only seen after the whole batch).
- Long waiting time for small jobs.

4. Multiprogramming Operating System

A **Multiprogramming Operating System** is an OS that allows **multiple programs to reside in memory at the same time** and **execute concurrently** by managing CPU time efficiently. While only **one process runs on the CPU** at a time, others wait in **ready or I/O queues**, improving **CPU utilization**.



Multiprogramming Operating System

OS keeps a pool of jobs in memory and schedules one after another.

No user interaction while program is running.

Uses **CPU scheduling** to select which job runs next.

Memory management is required to manage multiple jobs.

How Multiprogramming Works:

1. **Multiple programs are loaded into RAM** at the same time.
2. **The OS selects one program** to execute using a **CPU scheduling algorithm** (like FCFS, SJF, or Priority).
3. If the running program **needs I/O** (e.g., Reading Data from disk), it is moved to the **I/O waiting queue**.
4. The CPU is then assigned to the **next ready program** in the **ready queue**.
5. Once I/O is completed, the program is moved back to the **ready queue**.
6. This process **repeats** ensuring **maximum CPU efficiency**.

Advantages:

Better performance, it means high cpu utilization

Efficient use of resources.

Disadvantages:

Complex memory management & CPU scheduling.

Real-life example:

Web server handling many user requests simultaneously.

5. Time-Sharing Operating System (Multitasking)

Time-sharing operating system allows multiple users to use one system simultaneously.

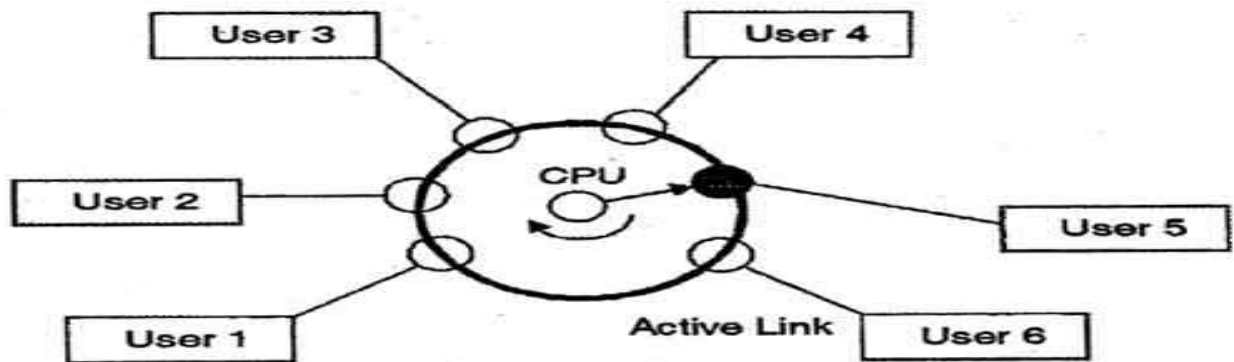
Each user gets a small amount of **time slice/ time quantum** of CPU.

System switches between users so quickly that each thinks they have full access.

Provides direct interaction between user and program.

Requires memory management to swap processes in/out quickly.

In a time-sharing system, the operating system manages the execution of multiple user programs, each of which runs for a short time slice, typically a few milliseconds to a few seconds. The system rapidly switches between different programs, giving the illusion to each user that they have exclusive access to the computer. The concept of time sharing system is shown in figure.



How Time-Sharing OS Works:

1. Multiple Jobs in Memory:

- The system loads several jobs (processes) into memory.
- These jobs can be from different users or multiple programs from the same user.

2. Scheduling (Using a Scheduler):

- The **CPU scheduler** picks one job from the ready queue based on a scheduling algorithm (like Round Robin).
- Each job is assigned a small time slice (time quantum), e.g., 10–100 milliseconds.

3. Execution & Time Slice:

- The selected job runs on the CPU for its allotted time.

- If the job finishes in that time, it leaves the queue.
- If not, it's moved to the back of the queue, and the next job gets CPU time.

4. **Context Switching:**

- When switching between jobs, the OS **saves the state** of the current job (program counter, registers, etc.).
- Then it **loads the state** of the next job to resume its execution.
- This is called a **context switch**.

5. **Repeat Cycle:**

- The OS continues this cycle, giving each job a fair share of CPU time.
- As new jobs arrive, they are added to the queue.

6. **User Interaction:**

- Users can interact with their programs (typing, input/output) almost in real time.
- The OS handles I/O and processing concurrently via time-sharing.

Advantages:

- Better **CPU utilization**
- Allows **interactive computing** (good for terminals)
- **Reduced idle time**
- Supports **multi-user** environments

Disadvantages:

- **Complexity** in managing memory and scheduling
- Not ideal for **real-time** systems where strict timing is required

6. **Real-Time Operating System (RTOS)**

A **Real-Time Operating System**: - is a type of software that runs on computers or devices and makes sure important tasks happen exactly on time — without delay.

Tasks are completed within strict time constraints (deadlines).

It mainly operates in **embedded systems** and **control systems**.

Two types:

- **Hard RTOS:** Tasks must be completed within fixed time.
- **Soft RTOS:** Timing is important but not strict (e.g., video streaming). It performs minimal swapping to ensure fast response.



1 Task Management (Multitasking)

- The RTOS runs **multiple tasks** (or threads) "simultaneously" using **context switching**.
- Each task has a **priority**, and the scheduler picks which task runs based on this.

2 Scheduler

- Determines which task runs next.
- Common types:
 - **Preemptive Scheduling:** A higher-priority task can interrupt a lower-priority one.

3 Deterministic Timing

- All operations, including task switching and interrupt handling, happen in **predictable, fixed time**.
- This ensures tasks meet their **deadlines**.

4 Interrupt Handling

- RTOS handles hardware **interrupts** (events from devices like sensors).
- Often assigns a **high-priority task** or **interrupt service routine (ISR)** to handle the event quickly.

5 Inter-task/process Communication & Synchronization

- Uses mechanisms like:

- **Semaphores** (signal-based task control)
- **Queues** (for passing messages)
- **Mutexes** (for resource protection)

6 Memory Management

- Usually very **lightweight** and **static** (less dynamic memory use) to avoid fragmentation and delays.
- **Applications:** Air traffic control, medical devices, radar, robotics.
Advantages: Fast response, high reliability.
Disadvantages: Complex to build, expensive, difficult to update.

7. Distributed Operating System

Manages multiple independent computers connected via network.

Looks like a single system to the user, though it's composed of many machines.

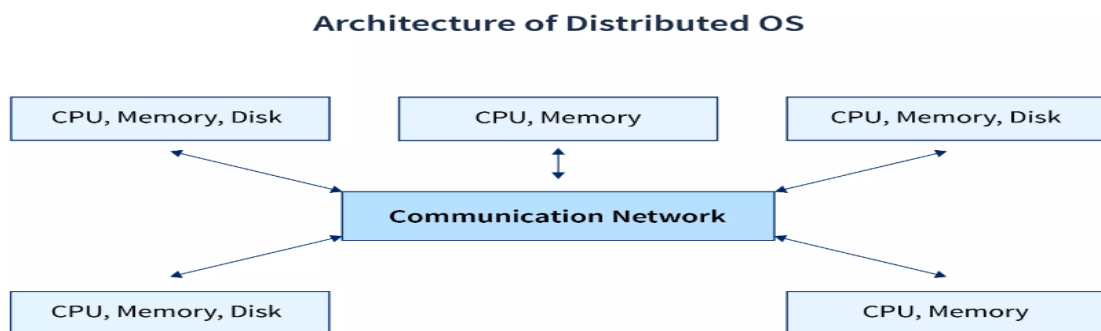
Each node has its own CPU and memory, and they share data & workload.

Communication is done via message passing.

Used in **peer-to-peer networks**, and **e-commerce**.

Offers **resource sharing, load balancing, and fault tolerance**.

If one system fails, others continue to work.



How It Works

1. Process Management

- Manages processes running on different machines.
- Supports process migration (move from one node to another).
- 2. **Communication**
 - Handles message-passing between nodes using protocols (like TCP/IP, RPC).
- 3. **File System**
 - Provides a **distributed file system** so files can be accessed from any node.
- 4. **Resource Management**
 - Allocates and monitors use of CPUs, memory, and I/O across the system.
- 5. **Naming**
 - Every resource has a **global name** (not tied to one machine).
- 6. **Security**
 - Manages access control and ensures secure communication.

Advantages of Distributed Operating System

1. **Resource Sharing** – Enables access to remote resources seamlessly.
2. **Fault Tolerance** – System continues functioning even if one node fails.
3. **Scalability** – Easily add more nodes to increase system capacity.
4. **Parallel Processing** – Tasks can run simultaneously across nodes.

Disadvantages of Distributed Operating System

1. **Complex Design** – Requires advanced architecture and algorithms.
2. **Security Risks** – More vulnerable due to multiple connected systems.
3. **Network Dependency** – Performance relies heavily on network quality.
4. **Software Compatibility** – Not all software works in a distribute

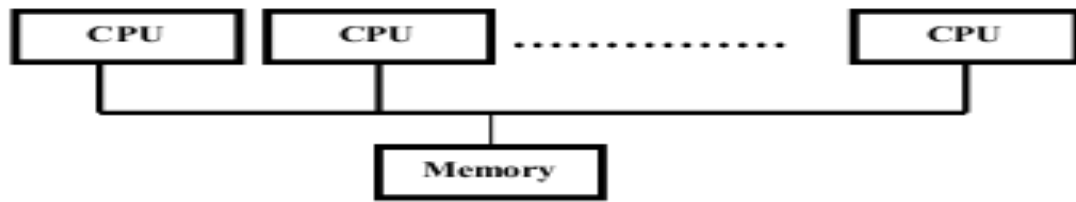
8. Parallel Operating System (Multiprocessor System)

Uses **two or more CPUs** in one system to increase processing speed.

All CPUs share memory, bus, and I/O devices.

Two types:

- **Symmetric (SMP):** All CPUs are equal and run the OS together.



Symmetric Multiprocessing Architecture

- **Asymmetric:** One master CPU controls others (slaves).
Improves system performance and reliability.
Used in **scientific simulations, large databases, real-time applications.**

Working Of Parallel Operating System :

1. Task Division

- The OS breaks a large problem or program into **smaller sub-tasks**.
- These tasks are designed to be **independent** or only lightly dependent.

2. Processor Assignment

- The OS assigns these sub-tasks to **different processors (or cores)** in the system.
- Each processor gets its own portion of work.

3. Parallel Execution

- All processors **work at the same time** on their assigned tasks.
- This is what gives the system **high speed and performance**.

4. Communication and Synchronization

- If tasks need to share data:
 - The OS uses **shared memory** (in SMP systems) or **message passing** (in cluster systems).
- It ensures **synchronization** to avoid conflicts (e.g., two tasks writing to the same data).

5. Result Integration

- Once all sub-tasks are finished, the OS **combines their results** into the final output.

Advantages

- **High Performance** – Executes tasks faster using multiple processors.
- **Efficient Resource Use** – Utilizes all CPUs fully for better output.
- **Scalability** – Can handle large workloads by adding more processors.

- **Faster Problem Solving** – Ideal for time-critical and complex computations.

Disadvantages

1. **Complex Programming** – Writing parallel programs is difficult.
2. **Synchronization Overhead** – Managing shared resources adds delays.
3. **Debugging Issues** – Hard to trace errors in concurrent execution.
4. **Hardware Cost** – Requires advanced multi-processor systems.

9. Operating System Components

These are the major components that help the OS manage resources:

1. Process Management

Manages creation, execution, suspension, and termination of processes.
It handles process scheduling based on priority, time, and state.
Ensures each process gets enough CPU time without interfering with others.
Supports multitasking, inter-process communication, and deadlock handling.

2. Memory Management

Allocates memory to processes and deallocates it when not needed.
Tracks memory usage using techniques like paging and segmentation.
Prevents memory leaks and ensures safe access by isolating process spaces.
Maximizes memory utilization and maintains system stability.

3. File System Management

Controls creation, deletion, access, and organization of files.
Manages file permissions, types, metadata, and directory structures.
Ensures data consistency through file locking and journaling.
Provides an interface for users and applications to store and retrieve data.

4. Device Management

Coordinates and monitors input/output devices like printers, disks, etc.
Uses device drivers as translators between hardware and software.
Handles queuing, buffering, and interrupt handling for efficient I/O.
Supports plug-and-play and dynamic device configuration.

5. Secondary Storage Management

Manages hard disks, SSDs, optical drives, and backup devices.
Allocates disk space and manages data placement and retrieval.
Monitors disk usage and provides tools for formatting and defragmentation.
Ensures long-term storage, reliability, and efficient access.

6. I/O System Management

Oversees all input and output operations between the CPU and peripherals.
Provides a uniform interface for I/O devices with buffering and spooling.
Handles device communication using interrupt-driven or DMA methods.
Ensures fast and synchronized data transfer with minimal CPU load.

7. Communication (Inter-process Communication - IPC)

Enables processes to communicate and share data securely and efficiently.
Supports pipes, message queues, shared memory, and sockets.
Allows synchronization using semaphores, mutexes, and signals.
Essential for parallel processing, client-server models, and distributed systems.

8. Protection and Security System

Prevents unauthorized access to data, memory, and hardware.
Implements access control, user authentication, and encryption.
Separates user space and kernel space to protect critical functions.
Monitors system behavior to detect and prevent attacks or misuse.

9. Command Interpreter (Shell)

Acts as the user interface to interact with the operating system.
Translates user commands into OS-level actions or system calls.
Supports scripting, automation, and command history features.
Can be CLI-based (like Bash) or GUI-based (like Windows Explorer).

10. Operating System Services

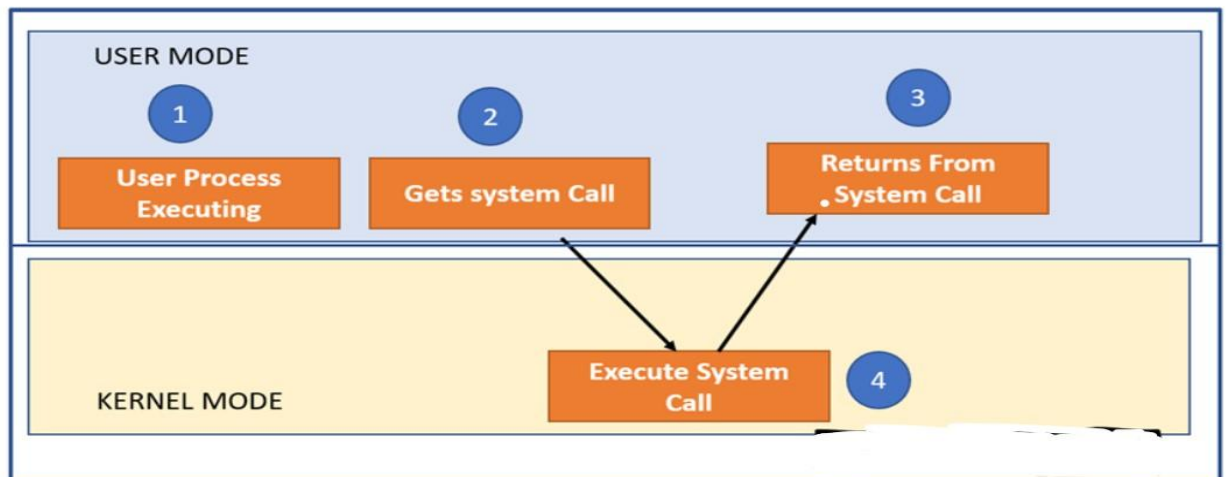
OS provides many essential services to both users and system programs:

1. **Process Management:** Scheduling, creating, terminating processes.
2. **Memory Management:** Allocates memory, handles virtual memory.
3. **File Management:** Reads, writes, and organizes files/directories.

4. **Device Management:** Coordinates device communication via drivers.
 5. **Network Management:** Manages network connectivity and protocols.
 6. **Security Management:** Handles access control, authentication.
 7. **User Interface:** GUI or CLI that allows users to interact with system.
- These services enhance usability, performance, and system integrity.

System Calls: System calls provide the interface between a running program and the operating system. They are generally available as assembly-language instructions.

System calls provide an interface between user-level processes and the operating system kernel, which is the core of the operating system that manages resources and provides services to applications. When a user-level process needs to perform an operation that requires kernel-level privileges, it makes a system call. The system call switches the process from user mode to kernel mode, where it can access system resources and perform the requested operation.



Architecture of the System Call

Different system calls are

1. Process control

end, abort load, execute create process, terminate process
get process attributes, set process attributes, wait for time, wait event
signal event

allocate and free memory

2. File management

create file, delete file

open, close, read, write, reposition

get file attributes, set file attributes

3. Device management

request device, release device

read, write, reposition

get device attributes, set device attributes

logically attach or detach devices

4. Information maintenance

get time or date, set time or date

get system data, set system data

get process, file, or device attributes

Process Management: in an operating system involves managing the creation, scheduling, execution, and termination of processes. When you run a program or application on your computer, it creates a process, which is a running instance of that program. The operating system manages these processes by allocating system resources such as CPU time, memory, and I/O devices, and ensuring that they run efficiently and without interfering with each other.

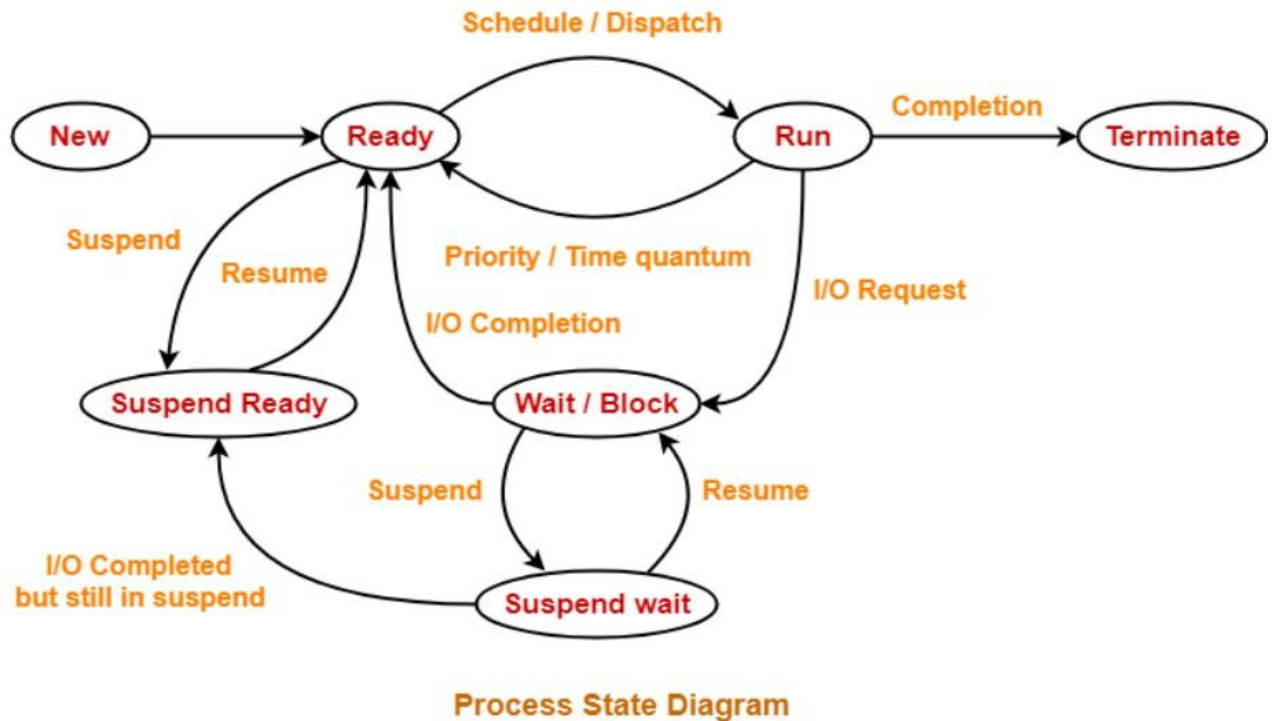
Definitions of Process: *Process is a program in execution*, and has its own memory space, system resources, and execution context. The operating system manages these processes by allocating resources such as CPU time, memory, and I/O devices, and ensuring that they run efficiently and without interfering with each other.

Process States: Process State:

A process executes it changes state. The state of a process is defined by the current activity of that process. Each process may be in one of the Following states:

1. **New:** The process is being created.
2. **Ready:** The process is waiting to be assigned to a processor.
3. **Running:** Instructions (processes) are being executed.

4. **Waiting:** The process is waiting for some event to occur (such as an I/O completion or response of a signal).



5. **Terminated:** The process has finished execution.
6. **Suspend ready** – Process that was initially in the ready state but was swapped out of main memory (refer Virtual Memory topic) and placed onto external storage by scheduler is said to be in suspend ready state. The process will transition back to ready state whenever the process is again brought onto the main memory.
7. **Suspend wait or suspend blocked** – Similar to suspend ready but uses the process which was performing I/O operation and lack of main memory caused them to move to secondary memory. When work is finished, it may go to suspend ready.

It is important to realize that only one process can be running on a processor at a time. Although many processes may be ready and waiting of processor.

Process State Transition: Here are some examples of state transitions:

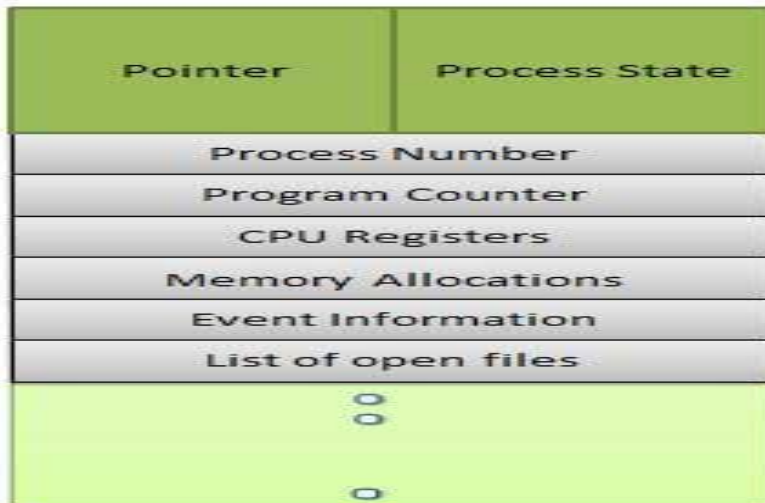
1. A process in the **new state** moves to the ready state when the operating system allocates system resources to it.
2. A process in the **ready state** moves to the running state when the CPU is available and the operating system selects it for execution.

3. A process in the **running state** moves to the blocked state when it requires some input/output operation to complete, such as reading data from a file or waiting for user input.
4. A process in the **blocked state** moves to the ready state when the event it was waiting for occurs, and the operating system re-enables the process for execution.
5. A process in the running state moves to the suspended state when the operating system temporarily suspends its execution to allocate resources to another process.
6. A process in the suspended state moves to the ready/suspended or blocked/suspended state when the process is resumed.

The operating system manages these state transitions to ensure that the processes are executed efficiently and without interference from other processes.

Process Control Block:

Each process is represented in the operating system by a **Process Control block (PCB)** also called as **Task Control Block**. A PCB is shown in Figure. It contains many pieces of information associated with a specific process, including these:



1. **Pointer:** Pointer points to another process control block. Pointer is used for maintaining the scheduling list.
2. **Process state:** The state may be new, ready running, waiting and so on.

3. **Program counter:** The counter indicates the address of the next instruction to be executed for this process.
4. **CPU Registers:** CPU registers include general purpose register, stack pointers, index registers and accumulator's etc. number of register and type of register totally depends upon the computer architecture. Along with the program counter, this state information must be saved when an interrupt occurs, to allow the process to be continued correctly after that process to process.
5. **CPU-scheduling information:** This information includes a process priority, pointers to scheduling queues, and any other scheduling parameters.
6. **Memory-management information:** This information may include such information as the value of the base and limit registers, the page tables, or the segment tables, depending on the memory system used by the operating system.
7. **Accounting information:** This information includes the amount of CPU and real time used, time limits, account numbers, job or process numbers, and so on.
8. **I/O status information:** This information includes the list of I/O devices allocated to the process, a list of open files, and so on.

In the PCB simply serves as the repository for any information that may vary from process to process.

Context switching:- in an operating system is the process of storing and restoring the state (or context) of a CPU so that execution can be resumed from the same point at a later time. This allows multiple processes to share a single CPU, appearing to run concurrently (a concept known as multitasking).

How It Works:

When the operating system switches from running one process to another, it needs to:

1. **Save the context** of the currently running process (e.g., CPU registers, program counter, stack pointer).
2. **Load the context** of the next process to run.

This saved information is typically stored in a data structure called the **Process Control Block (PCB)**.

Operations on Processes: Operating systems provide various operations to manage processes. Here are some of the common operations on processes:

1. **Process Creation:** The operating system creates a new process when a user requests to start a new program or process.
2. **Process Termination:** When a process finishes executing, or is no longer needed, the operating system terminates the process.
3. **Process Scheduling:** The operating system determines which process to execute next based on scheduling algorithms and process priorities.
4. **Process Synchronization:** Operating systems provide synchronization mechanisms to prevent race conditions and ensure that multiple processes access shared resources in a mutually exclusive manner.
5. **Process Monitoring:** The operating system tracks the performance of running processes, such as CPU usage, memory usage, and I/O operations.
6. **Process Resource Allocation:** The operating system allocates resources, such as CPU time, memory, and I/O devices, to processes based on their requirements and priorities.
7. **Process Suspension/Resumption:** The operating system can suspend a process temporarily to free up resources for other processes and resume it later.